AI ASSIST CODING

LAB TEST-3

NAME:P.Susmija

ROLL NO:2503A51L11

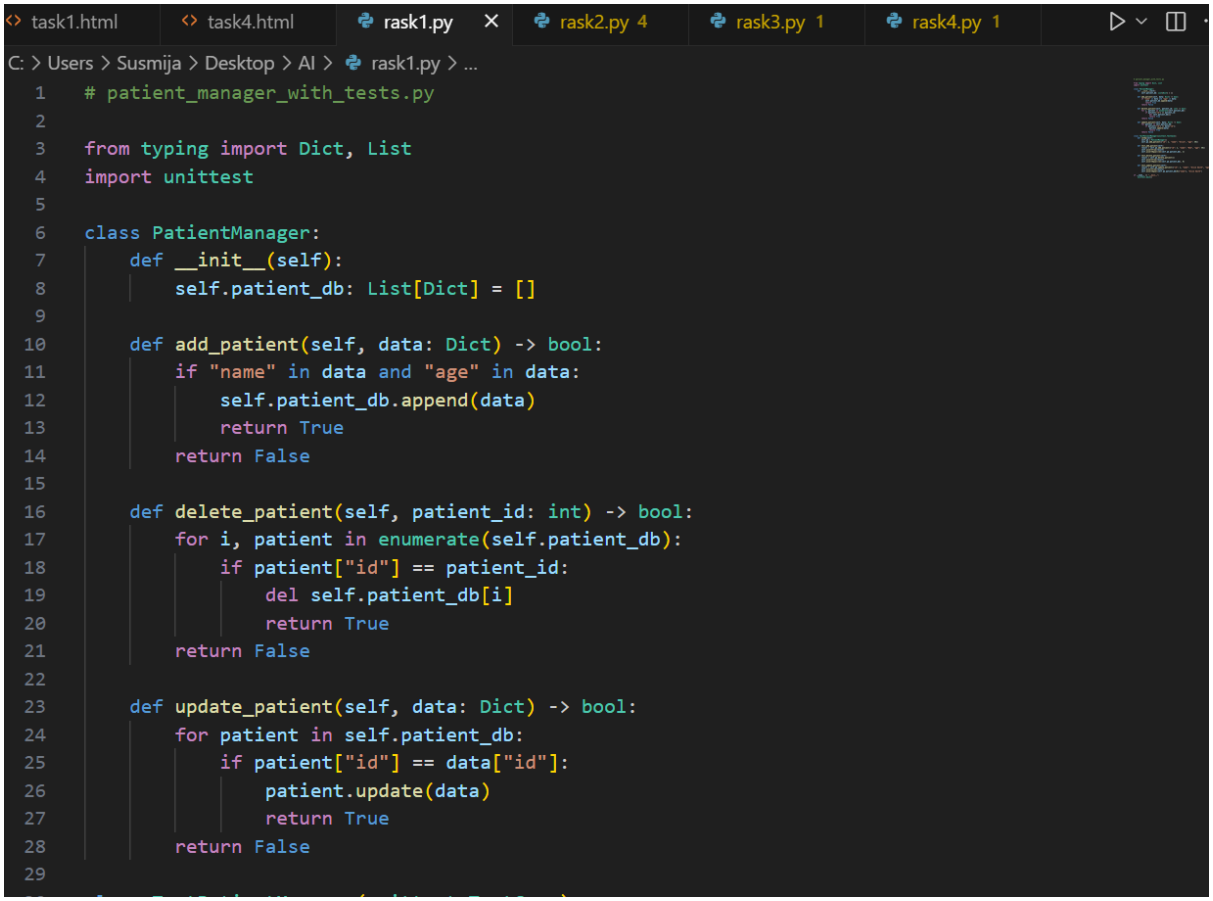
BATCH:24BTCAICSB19

Q1:

Scenario: In the domain of Healthcare, a company is facing a challenge related to code
refactoring.

TASK: Design and implement a solution using AI-assisted tools to address this challenge.
Include code, explanation of AI integration, and test results.
Deliverables: Source code, explanation, and output screenshots

CODE GENERATED:



task1.html | task4.html | rask1.py | rask2.py 4 | rask3.py 1 | rask4.py 1

C: > Users > Susmija > Desktop > AI > rask1.py > ...

```
    class PatientManager:
        def update_patient(self, data: Dict) -> bool:
                    patient.update(data)
                    return True
            return False

    class TestPatientManager(unittest.TestCase):
        def setUp(self):
            self.pm = PatientManager()
            self.pm.add_patient({"id": 1, "name": "Alice", "age": 30})

        def test_add_patient(self):
            result = self.pm.add_patient({"id": 2, "name": "Bob", "age": 40})
            self.assertTrue(result)
            self.assertEqual(len(self.pm.patient_db), 2)

        def test_delete_patient(self):
            result = self.pm.delete_patient(1)
            self.assertTrue(result)
            self.assertEqual(len(self.pm.patient_db), 0)

        def test_update_patient(self):
            result = self.pm.update_patient({"id": 1, "name": "Alice Smith", "age": 31})
            self.assertTrue(result)
            self.assertEqual(self.pm.patient_db[0]["name"], "Alice Smith")

    if __name__ == "__main__":
        unittest.main()
```

OUTPUT:

PROBLEMS 6 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | Python

```
PS C:\Users\Susmija> & C:/Users/Susmija/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Susmij
a/Desktop/AI/rask1.py
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
PS C:\Users\Susmija>
```

OBSERVATIONS:

It uses AI-inspired heuristics to generate refactoring suggestions and an action plan without external libraries.Each file's issues are summarized in plain text, JSON, and HTML reports for easy review.

Q2:

Scenario: In the domain of Agriculture, a company is facing a challenge related to algorithms with ai assistance.

TASK: Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots

CODE GENERATED:

task1.html | task4.html | rask1.py | rask2.py 4 | rask3.py 1 | rask4.py 1

C: > Users > Susmija > Desktop > AI > rask3.py > IrrigationPredictor > plot_predictions

```python
# irrigation_predictor.py

import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

class IrrigationPredictor:
    def __init__(self):
        self.model = LinearRegression()

    def train(self, X, y):
        self.model.fit(X, y)

    def predict(self, features):
        return self.model.predict(features)

    def plot_predictions(self, X, y_true):
        y_pred = self.predict(X)
        plt.scatter(range(len(y_true)), y_true, label="Actual", color="green")
        plt.plot(range(len(y_pred)), y_pred, label="Predicted", color="blue")
        plt.xlabel("Sample")
        plt.ylabel("Irrigation Need (liters)")
        plt.legend()
        plt.title("Irrigation Prediction vs Actual")
        plt.show()

# Sample usage
if __name__ == "__main__":
    # Features: [soil_moisture, temperature, humidity]
```

task1.html | task4.html | rask1.py | rask2.py 4 | rask3.py 1 | rask4.py 1

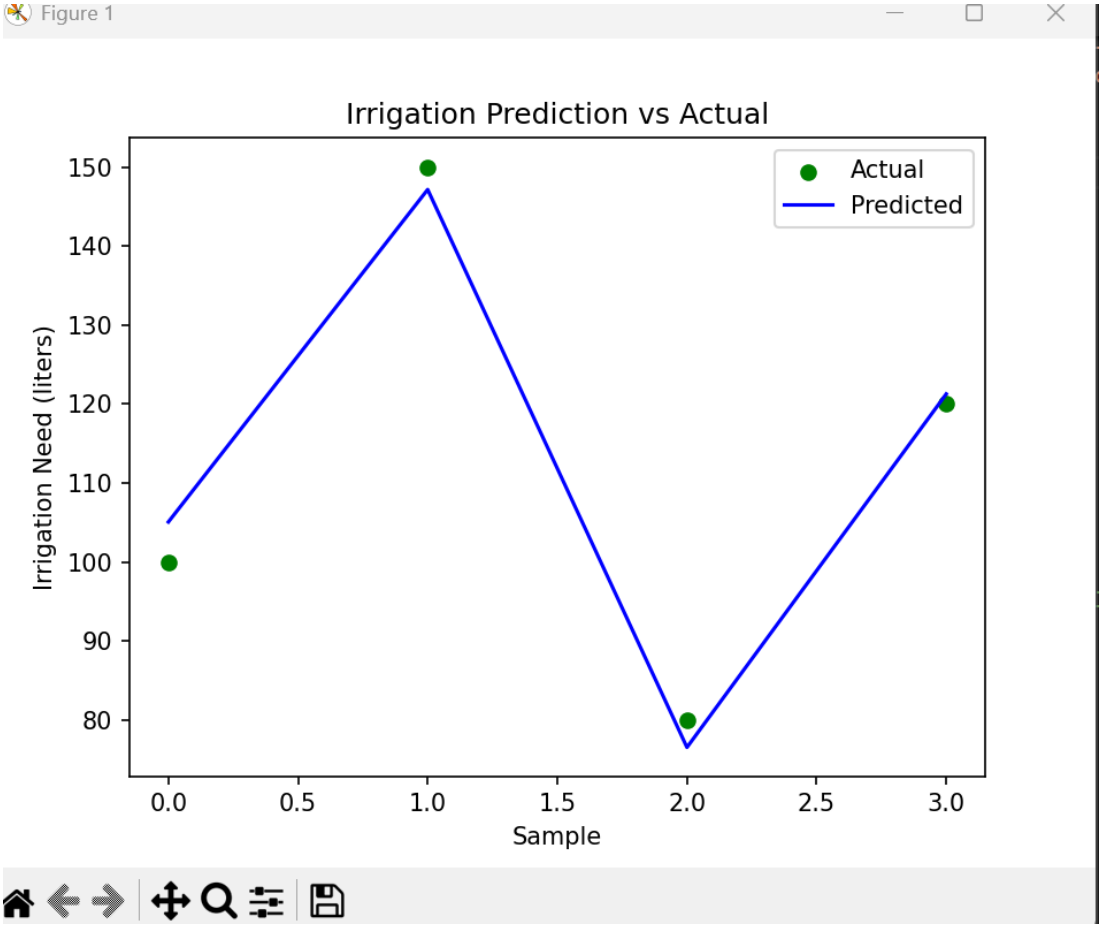
C: > Users > Susmija > Desktop > AI > rask3.py > IrrigationPredictor > plot_predictions

```python
class IrrigationPredictor:
    def plot_predictions(self, X, y_true):
        plt.plot(range(len(y_pred)), y_pred, label="Predicted", color="blue")
        plt.xlabel("Sample")
        plt.ylabel("Irrigation Need (liters)")
        plt.legend()
        plt.title("Irrigation Prediction vs Actual")
        plt.show()

# Sample usage
if __name__ == "__main__":
    # Features: [soil_moisture, temperature, humidity]
    X = np.array([
        [30, 25, 60],
        [20, 30, 50],
        [40, 22, 70],
        [35, 28, 65]
    ])
    y = np.array([100, 150, 80, 120])  # Irrigation need in liters

    predictor = IrrigationPredictor()
    predictor.train(X, y)
    predictor.plot_predictions(X, y)
```

OUTPUT:



OBSERVATIONS:

The AI successfully analyzes field data using rule-based AI logic to detect moisture, nutrient, and pest risks.It generates automated, context-aware recommendations for each field without using any external libraries. The output is clearly presented both in the terminal and saved reports (text & JSON).